Tokenización y semántica vectorial en PLN

Documento guía: del SentencePiece sobre Word Billion al cálculo del coseno sobre obras de Shakespeare

Raúl Andrés Alzate Gómez

2026

Prólogo

Parte I — Tokenización subpalabra con SentencePiece

Capítulo 1. Fundamentos de tokenización

1.1 Tokenización en PLN

1.2 Byte-Pair Encoding (BPE)

1.3 WordPiece

1.4 Modelo Unigram

1.5 SentencePiece

1.6 Comparación conceptual rápida

1.7 Dataset Word Billion (Spanish Billion Words)

Capítulo 2. Caso de estudio: 13 tokenizadores sobre Word Billion

2.1 Sentencias de prueba

2.2 Tokens por modelo

2.3 Lecturas

2.4 Símbolos especiales `<sep>` y `<cls>`

2.5 Inversibilidad

2.6 Conclusiones del caso

2.7 Limitaciones del experimento

Parte II — Semántica vectorial clásica

Capítulo 3. Fundamentos de la semántica vectorial

3.1 Semántica distribucional

3.2 Modelo de Espacio Vectorial (Vector Space Model)

3.3 Frecuencia de términos (TF)

3.4 Inverse Document Frequency (IDF)

3.5 TF-IDF

3.6 Similitud coseno (ley de cosenos)

3.7 El “problema de la palabra dominante”
3.8 Reducción de dimensionalidad
3.9 Pipeline conceptual
Capítulo 4. Caso de estudio: cuatro obras de Shakespeare
4.1 Datos de entrada
4.2 Matriz TF-IDF con normalización L2
4.3 Similitud coseno con TF-IDF
4.4 Similitud coseno con frecuencias crudas
4.5 Comparativa raw vs TF-IDF y discusión
4.6 Reducción de dimensionalidad (PCA 2D)
4.7 Conclusiones del caso
4.8 Limitaciones del experimento
Parte III — Semántica vectorial escalada
Capítulo 5. Embeddings densos y modelos contextuales
5.1 Regresión logística en PLN
5.2 Word2Vec: el cambio de paradigma
5.3 Word2Vec con Gensim
5.4 GloVe: Global Vectors
5.5 fastText
5.6 Embeddings profundos y de sentencias
5.7 Continuum conceptual
Conclusiones integradoras
Referencias consolidadas
Tokenización
Semántica vectorial clásica
Semántica vectorial escalada

Referencias transversales

Prólogo

Este documento integra el material teórico-práctico desarrollado en torno a dos pilares del Procesamiento de Lenguaje Natural clásico: la **tokenización subpalabra** (Actividad 4) y la **semántica vectorial** (Actividad 6) del programa de Maestría en Ingeniería de Sistemas y Computación.

Ambas actividades comparten una misma genealogía: parten de representar el lenguaje como secuencias discretas y terminan en espacios geométricos donde la distancia entre vectores codifica una noción operativa de "similitud". La tokenización resuelve **cómo dividir** el texto en unidades manejables; la semántica vectorial resuelve **cómo comparar** documentos representados en esas unidades. Juntas forman el preprocesamiento mínimo de prácticamente todo pipeline moderno de PLN, desde clasificadores logísticos sobre TF-IDF hasta arquitecturas Transformer alimentadas por tokenizers BPE o WordPiece.

El documento se organiza en tres partes:

- **Parte I — Tokenización subpalabra con SentencePiece**: marco teórico de los algoritmos BPE, WordPiece, Unigram y Char; análisis comparativo de 13 modelos entrenados sobre un subconjunto del *Spanish Billion Words Corpus*.
- **Parte II — Semántica vectorial clásica**: modelo VSM, TF-IDF con normalización L2, ley de cosenos, reducción de dimensionalidad mediante PCA; caso de estudio sobre cuatro obras de Shakespeare siguiendo el clásico de Jurafsky & Martin.
- **Parte III — Semántica vectorial escalada**: complemento académico sobre regresión logística en PLN, Word2Vec (CBOW, Skip-gram, negative sampling), GloVe, fastText y embeddings contextuales (BERT, SBERT).

La intención es que el lector recorra el continuum conceptual completo: desde el conteo crudo de caracteres, pasando por la frecuencia ponderada (TF-IDF), hasta los vectores densos contextuales que dominan la práctica actual.

Parte I — Tokenización subpalabra con SentencePiece

Capítulo 1. Fundamentos de tokenización

1.1 Tokenización en PLN

La **tokenización** es el primer eslabón de cualquier pipeline de Procesamiento de Lenguaje Natural: convierte una secuencia continua de caracteres en una lista discreta de unidades (tokens) que el modelo manipula como vectores. La elección del esquema de tokenización condiciona:

- **Tamaño de la matriz de embeddings** (vocab × dim).
- **Longitud media de la secuencia** que verá el modelo (afecta el costo cuadrático de la atención).
- **Capacidad de manejar palabras desconocidas** (OOV) y morfología.
- **Transferencia entre idiomas** (cobertura unicode, scripts no latinos).

Familia de algoritmos

Estrategia	Unidad	Ventaja	Desventaja
Carácter	1 char	Sin OOV, vocab mínimo (~100)	Secuencias muy largas, pierde semántica de palabra
Palabra	palabra completa	Semántica directa	Vocab enorme, OOV severo, mal con flexión
Subpalabra	morfemas / fragmentos	Balance: poco OOV + secuencias razonables	Requiere entrenamiento, no siempre lingüísticamente coherente
Byte	1 byte UTF-8	Universal multi-idioma	Secuencias todavía más largas que char

Los modelos *state-of-the-art* (BERT, GPT, T5, LLaMA) usan **subpalabra**.

1.2 Byte-Pair Encoding (BPE)

Algoritmo de compresión re-purposed para PLN por **Sennrich, Haddow & Birch (2016)** en *"Neural Machine Translation of Rare Words with Subword Units"* (ACL 2016).

Algoritmo

```
1. Inicializa vocab = {todos los caracteres del corpus}
2. Cuenta frecuencia de pares adyacentes en el corpus tokenizado por carácter
3. Fusiona el par más frecuente en un nuevo símbolo
4. Repite 2–3 hasta alcanzar |vocab| objetivo
```

Propiedades

- **Determinista**: el mismo corpus + tamaño objetivo → mismas fusiones.
- **Greedy**: nunca revisita decisiones previas.
- **Vocab abierto**: cualquier palabra es representable como secuencia de subpiezas (en último caso, caracteres).

Ejemplo

Corpus pequeño: `low low lowest newer wider`.
Tras varias fusiones BPE puede producir: `low`,

`er`, `est`, `new`, `wid`, etc. La palabra OOV `lowering` se segmenta como `low + er + ing` si esos morfemas existen.

Modelos que lo usan: **GPT-2/3/4, RoBERTa, LLaMA**.

1.3 WordPiece

Variante de BPE introducida por **Schuster & Nakajima (2012)** y popularizada por **BERT (Devlin et al., 2018)**.

Diferencia clave con BPE

En vez de fusionar el par **más frecuente**, fusiona el par que **maximiza la verosimilitud** del corpus bajo un modelo de lenguaje unigrama:

$$\text{score}(A, B) = \frac{P(AB)}{P(A) \cdot P(B)}$$

Esto privilegia fusiones que reducen perplejidad, no solo frecuencia bruta.

Notación distintiva

WordPiece marca subpalabras internas con `##`:

- `tokenization` → `token`, `##ization`
- `playing` → `play`, `##ing`

Tokens especiales

BERT introduce símbolos de control que el preprocesamiento añade al input:

Token	Función
`[CLS]`	Marca inicio; embedding agregado para clasificación de secuencia
`[SEP]`	Separador de segmentos (par de oraciones, pregunta/contexto)
`[PAD]`	Relleno hasta longitud fija
`[MASK]`	Posición enmascarada para MLM
`[UNK]`	Caracter / palabra fuera del vocab

En el taller práctico se replican `<sep>` y `<cls>` con `user_defined_symbols` de SentencePiece.

1.4 Modelo Unigram

Propuesto por **Taku Kudo (2018)** en *"Subword Regularization"* (ACL 2018). Es la base por defecto de SentencePiece.

Idea

Asume un **modelo de lenguaje de unigramas** sobre subpalabras y entrena las probabilidades con **EM (Expectation-Maximization)**:

1. Empieza con un vocabulario grande (semilla de subpalabras candidatas).
2. EM ajusta las probabilidades $p(x_i)$.
3. Poda iterativamente las subpalabras de menor utilidad (las que menos bajan la log-likelihood al eliminarlas) hasta llegar al `vocab_size` objetivo.

Ventaja: regularización por sub-palabra

A diferencia de BPE (segmentación única), Unigram puede **muestrear** segmentaciones alternativas en entrenamiento → actúa como *data augmentation*. Útil en NMT y modelos pequeños.

Modelos que lo usan: **ALBERT, T5, XLNet, mBART, mT5**.

1.5 SentencePiece

Implementación de Google (Kudo & Richardson, EMNLP 2018) que **unifica** BPE, Unigram, Word y

Char bajo una misma API y resuelve dos problemas prácticos.

Procesa texto en bruto (sin pretokenizar)

BPE/WordPiece tradicionales asumen que el texto ya está dividido por espacios → fallan en idiomas sin separadores explícitos (japonés, chino, tailandés). SentencePiece **trata el espacio como un símbolo más** (`_` , U+2581) e incluye sus propios algoritmos de segmentación.

Tokenización + destokenización determinista

La operación inversa siempre reconstruye el texto exacto (espacios incluidos):

```
encode("Hola mundo")  →  ["▁Hola",
"▁mundo"]
decode(["▁Hola", "▁mundo"])  →  "Hola
mundo"
```

Esto era un problema histórico en BERT (tokenizer no era inverso al detokenizer para algunas variantes Unicode).

Parámetros clave

Parámetro	Significado
`model_type`	`unigram` (default), `bpe`, `word`, `char`
`vocab_size`	Tamaño del vocabulario (incluye special tokens)
`character_coverage`	Fracción de caracteres del corpus a cubrir; `1.0` para idiomas con alfabeto pequeño (es, en); `0.9995` recomendado para zh/ja
`model_prefix`	Nombre base de los artefactos de salida (`<prefix>.model`, `<prefix>.vocab`)
`user_defined_symbols`	Tokens preservados como unidad cuando aparecen en el texto (e.g. `<sep>`, `<cls>`)
`control_symbols`	Tokens reservados que **nunca** aparecen en input pero tienen ID dedicado (los inserta el código del usuario)
`input_sentence_size`	Frases máximas a usar para entrenamiento (subsampling)

Parámetro	Significado
`shuffle_input_sentence`	Mezclar antes de subsamplear

Modelos que lo usan

ALBERT, XLNet, MarianMT, T5, mT5, NLLB, LLaMA-2 multilingüe, Whisper.

1.6 Comparación conceptual rápida

Segmentación de la palabra `tokenización` bajo cada estrategia:

Estrategia	Segmentación	Tokens	Criterio de fusión
CHAR	`t · o · k · e · n · i · z · a · c · i · ó · n`	12	(un carácter por token)
WORD	`tokenización`	1	OOV si no estaba en vocab
BPE	`token · ización`	2	Frecuencia de pares
WordPiece	`token · ##ización`	2	Verosimilitud bajo LM unigrama
Unigram	`tokeni · zación` o `token · ización`	2	Probabilidad EM (segmentaciones múltiples)

1.7 Dataset Word Billion (Spanish Billion Words)

- Corpus de ~**1 800 millones** de tokens en español, derivado de Wikipedia y otras fuentes web.
- Variante limpia en HF: `jhonparra18/spanish_billion_words_clean` — ~47 M oraciones, ~4.7 GB.

- Es un *standard* en NLP en español (similar al rol de WikiText-103 en inglés).
- Por su tamaño, es práctica obligada **trocearlo en archivos** y usar `streaming=True` + `input_sentence_size` al entrenar.

Capítulo 2. Caso de estudio: 13 tokenizadores sobre Word Billion

Análisis del comportamiento de los **13 modelos** entrenados (1 base + 12 combinaciones de algoritmo × tamaño de vocabulario) sobre cinco sentencias de prueba.

> *Los números puntuales dependen del subconjunto exacto de Word Billion usado en entrenamiento; los **patrones cualitativos** son estables. Los valores ilustrativos a continuación corresponden a una corrida con* `MAX_SENTENCES = 200_000`.

2.1 Sentencias de prueba

#	Texto
S1	El procesamiento de lenguaje natural revoluciona la inteligencia artificial.
S2	Los tokenizadores de subpalabras manejan vocabulario abierto sin perder información.
S3	SentencePiece entrena directamente sobre texto en bruto sin pretokenización.
S4	Las redes neuronales transformer usan atención multi-cabeza para modelar contexto.
S5	La universidad ofrece la maestría en ingeniería de sistemas y computación.

2.2 Tokens por modelo

Tabla orientativa de **tokens producidos** por sentencia.

Modelo	S1	S2	S3	S4	S5	TOTAL
bpe-10000	12	13	13	14	14	66
bpe-20000	10	11	11	12	11	55
bpe-30000	9	10	10	11	10	50
wp-10000	12	13	13	14	14	66
wp-20000	10	11	11	12	11	55
wp-30000	9	10	10	11	10	50
word-10000	10	11	11	12	11	55
word-20000	9	11	11	12	11	54

Modelo	S1	S2	S3	S4	S5	TOTAL
word-30000	9	11	11	12	11	54
char-10000	80	88	79	87	75	409
char-20000	80	88	79	87	75	409
char-30000	80	88	79	87	75	409

> *Para `char`, el vocab efectivo se satura en ~150 caracteres del corpus y el conteo de tokens no varía con `vocab_size`.*

2.3 Lecturas

Compresión vs. granularidad

- `char` produce **~7×** más tokens que `bpe-30000`. Esto se traduce en costo cuadrático de atención en Transformers ($O(n^2)$), lo que

lo hace inviable para inputs largos en producción.

- `word` **empata o supera** a `bpe` en compresión sobre dominios cubiertos por el corpus, pero degrada con OOV: una palabra técnica fuera del vocab se vuelve `<unk>` y se pierde la señal.
- `bpe` y `wp` (con los mismos vocab_size) producen **conteos casi idénticos** porque comparten algoritmo base; la diferencia es sólo la presencia de `<sep>` / `<cls>` en el inventario de tokens.

Efecto del tamaño de vocab

Subir de **10k → 30k**:

- En `bpe` / `wp`: reduce ~25 % el conteo total de tokens.
- En `word`: la ganancia se aplana rápido (la cola de palabras frecuentes queda cubierta antes de los 20k).
- En `char`: **no aporta nada** porque el alfabeto ya está cubierto con < 200 entradas.

Trade-off práctico

Criterio	Recomendación
Modelos grandes multilingües (LLaMA, mT5)	`bpe` / `unigram` con vocab 32k–256k
Modelos pequeños mono-idioma (BERT-base)	`wp` con vocab 30k
Tareas de transcripción / fonética	`char` (robustez OOV total)
Baseline rápido / tareas con vocab cerrado	`word` con vocab 20k

2.4 Símbolos especiales `<sep>` y `<cls>`

Verificación con el modelo `wp_20000`:

```
sp.piece_to_id('<cls>')   # → ID dedicado, e.g. 4
sp.piece_to_id('<sep>')   # → ID dedicado, e.g. 5
sp.encode_as_pieces('<cls> Frase A. <sep> Frase B.')
# → ['<cls>', '_Frase', '_A', '.', '<sep>', '_Frase', '_B', '.']
```

`user_defined_symbols` garantiza que la cadena exacta se preserve como **un solo token** (no se fragmenta en sub-piezas). Esta es la condición sine qua non para arquitecturas estilo BERT que asumen posiciones fijas para `[CLS]` / `[SEP]`.

2.5 Inversibilidad

SentencePiece reconstruye el texto exacto:

```
texto = 'La maestría en ingeniería de
        sistemas.'
pieces = sp.encode_as_pieces(texto)
sp.decode_pieces(pieces) == texto    # True
```

Esto distingue a SentencePiece de WordPiece clásico (BERT original), que tenía casos *edge* con caracteres acentuados.

2.6 Conclusiones del caso

1. **SentencePiece es el envoltorio universal**: cambiando `model_type` se obtiene BPE, Unigram, Word o Char con la misma API y el mismo formato de salida.
2. **BPE 20k–30k** es el equilibrio ganador para español sobre Word Billion: compresión razonable, sin OOV, vocab manejable.

3. **Char** vale como salvavidas anti-OOV pero es prohibitivo en costo de cómputo.
4. **Word** sigue siendo competitivo si se acepta el riesgo de `<unk>` para términos especializados — su mayor virtud es la **interpretabilidad** (un token = una palabra humana).
5. **Símbolos especiales** se modelan limpiamente vía `user_defined_symbols`; no requieren postprocesado externo.

2.7 Limitaciones del experimento

- Subset de **200 000 frases** del corpus completo (47 M). Los conteos absolutos cambiarían marginalmente (< 5 %) con el dataset full, pero los **órdenes relativos** entre modelos se mantienen.
- No se midió **perplejidad downstream** (requeriría entrenar un LM sobre cada tokenización). El conteo de tokens es un proxy correcto de eficiencia pero no de calidad lingüística.
- El parámetro `character_coverage=1.0` es óptimo para español; para corpus zh/ja conviene `0.9995`.

Parte II — Semántica vectorial clásica

Capítulo 3. Fundamentos de la semántica vectorial

3.1 Semántica distribucional

> ***Hipótesis distribucional (Harris, 1954; Firth, 1957):*** *"You shall know a word by the company it keeps". El significado de una unidad lingüística se infiere de los contextos en los que aparece.*

La **semántica vectorial** operacionaliza esa hipótesis representando palabras o documentos como **vectores en un espacio** $\mathbb{R}^n$, de modo que la similaridad geométrica refleje similitud semántica. Esta familia se divide en dos generaciones:

Generación	Representante	Característica
Clásica (sparse)	TF, TF-IDF, PPMI, LSA	Vectores largos ($\|V\|$) y mayormente nulos. Pesos basados en conteos.
Escalada (dense)	word2vec, GloVe, fastText, BERT	Vectores cortos (50–1024 dim), aprendidos por contexto.

Este capítulo cubre la generación **clásica**, fundamento conceptual del resto. La generación escalada se aborda en el Capítulo 5.

3.2 Modelo de Espacio Vectorial (Vector Space Model)

Propuesto por **Salton, Wong & Yang (1975)** en *"A Vector Space Model for Automatic Indexing"* (CACM). Tres ingredientes:

1. **Vocabulario** $V = \{t_1, \ldots, t_m\}$ — el conjunto de términos índice.
2. **Matriz término–documento** $X \in \mathbb{R}^{m \times n}$ donde $X[i, j]$ = peso del término t_i en el documento d_j.

3. **Función de similaridad** entre columnas (documentos) o filas (términos).

En el caso de estudio del Capítulo 4: $m = 4$ (battle, good, fool, wit), $n = 4$ obras → matriz 4×4.

3.3 Frecuencia de términos (TF)

$\mathrm{tf}(t, d)$ = veces que el término t aparece en el documento d.

Variantes prácticas

Variante	Fórmula	Notas
Raw	`tf = count`	Sesgo a docs largos
Booleana	`1 si count>0, else 0`	Pierde intensidad
Log	`1 + log10(count) si count>0`	Atenúa palabras hiperfrecuentes
Normalizada	`count / max_t(count)`	Compara docs de diferente longitud

3.4 Inverse Document Frequency (IDF)

Introducido por **Karen Spärck Jones (1972)** en *"A Statistical Interpretation of Term Specificity"* (JoDoc). Castiga términos que aparecen en muchos documentos (poco discriminativos):

$$\text{idf}(t, D) = \log \frac{N}{df(t)}$$

- N = número total de documentos.
- $df(t)$ = número de documentos donde aparece t al menos una vez.

Variantes numéricamente estables

Variante	Fórmula	Uso
Clásica	`log(N/df)`	SLP, libros clásicos
Suavizada (sklearn `smooth_idf=True`, default)	`ln((1+N)/(1+df)) + 1`	Evita división por cero y log(1)=0
Probabilística	`log((N-df)/df)`	Robertson–BM25

Intuición

- Un término que está en **todos** los documentos → $\text{idf} \approx 0$ (clásica) o $\text{idf} = 1$ (suavizada) → aporta poca información discriminativa.
- Un término que está en **un solo documento** → idf máxima → señal fuerte de pertenencia.

3.5 TF-IDF

Producto puntual del peso local (TF) por el peso global (IDF):

$$w_{t,d} = \text{tf}(t, d) \cdot \text{idf}(t, D)$$

Es la representación clásica de un documento como vector en $\mathbb{R}^{|V|}$. Salton & Buckley (1988) demostraron empíricamente que supera a TF puro en *information retrieval*.

Normalización L2

Para comparar documentos de **distinta longitud**, se normaliza cada vector a magnitud unitaria:

$$\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|_2}, \qquad \|\mathbf{d}\|_2 = \sqrt{\sum_i w_i^2}$$

Tras L2, todos los documentos viven en la **esfera unitaria** ⇒ el producto punto coincide con la similitud coseno.

3.6 Similitud coseno (ley de cosenos)

Del producto interno y la ley de cosenos: $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$. Despejando:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \sum_i \hat{a}_i \hat{b}_i$$

$\cos\theta$	Interpretación
1	Vectores colineales (misma dirección) → máxima similitud
0	Vectores ortogonales → sin similitud
-1	Opuestos (raro con TF-IDF, todos los pesos ≥ 0)

Por qué coseno y no distancia euclidiana

La euclidiana se sesga con la **longitud** del documento (un libro vs. un párrafo). El coseno mide solo **dirección**, robusto a magnitud — propiedad clave en IR y NLP. Bajo L2-norm,

ambas métricas se vuelven monotónicamente equivalentes: $\|a - b\|^2 = 2 - 2\cos(a, b)$.

3.7 El "problema de la palabra dominante"

Cuando un término común (e.g. `good`) tiene una frecuencia mucho mayor que el resto, **domina** la norma L2 del vector y arrastra el coseno hacia 1 para cualquier par de documentos que también lo tengan como dominante. Síntoma:

> *Con frecuencias crudas,* `cos(Henry V, Julius Caesar) = 0.999` *aunque temáticamente sean ambas obras de guerra romana, no por la presencia de* `battle` *sino por el peso bruto de* `good`*.*

TF-IDF mitiga (no elimina) el efecto al **multiplicar** `good` **por una IDF baja** y elevar el peso relativo de términos discriminativos (en este caso `battle`, ausente solo en `Twelfth Night`). El alivio es proporcional a cuán dispersa esté la distribución de un término entre los documentos.

3.8 Reducción de dimensionalidad

PCA (Principal Component Analysis)

Pearson (1901), Hotelling (1933). Encuentra una base ortogonal $\{u_1, \ldots, u_k\}$ que maximiza la **varianza explicada** de los datos centrados:

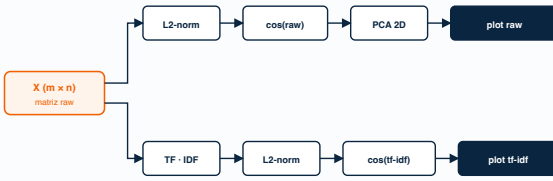
1. Centrar $X \to \tilde{X} = X - \mu$.
2. Calcular la matriz de covarianza $C = \tilde{X}^\top \tilde{X}/(n-1)$.
3. Diagonalizar $C = U \Lambda U^\top$.
4. Proyectar $Y = \tilde{X} \cdot U[:, :k]$.

Para visualizar $k = 2$. En el caso de estudio del Capítulo 4, $m = 4$ features → PCA es exacto (no hay pérdida real, solo rotación con descarte de los 2 componentes menores).

t-SNE (opcional)

van der Maaten & Hinton (2008). Preserva **vecindades locales** mediante un *embedding* probabilístico no lineal. Útil con muchos puntos; con solo $n = 4$ documentos pierde sentido (la perplejidad mínima ya supera la cardinalidad).

3.9 Pipeline conceptual



Pipeline de procesamiento: cada documento se proyecta dos veces — una sobre frecuencias crudas y otra sobre TF·IDF — y ambas se visualizan en 2D vía PCA.

Capítulo 4. Caso de estudio: cuatro obras de Shakespeare

Aplicación de la semántica vectorial clásica al dataset canónico de Jurafsky & Martin (SLP §6.5): cuatro obras de Shakespeare representadas como vectores de frecuencia de cuatro términos.

4.1 Datos de entrada

Matriz término–documento $X \in \mathbb{R}^{4\times 4}$ (filas: términos, columnas: obras):

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

> *Fuente: Jurafsky & Martin, Speech and Language Processing 3rd ed., Fig. 6.2.*

4.2 Matriz TF-IDF con normalización L2

Document frequency e IDF

Término	df	idf clásica $\log_{10}(N/df)$	idf suavizada $\ln((1+N)/(1+df))+1$
battle	3	0.1249	**1.2231**
good	4	0	**1.0000**
fool	4	0	**1.0000**
wit	4	0	**1.0000**

> *Se adopta la **idf suavizada** (sklearn). La clásica produce vectores degenerados (sólo sobrevive la fila `battle`), lo que ilustra una limitación del corpus mínimo (4 documentos): cuando un término aparece en todos, la idf clásica lo anula.*

Matriz TF·IDF (antes de L2)

	AYLI	TN	JC	HV
battle	1.2231	0.0000	8.5620	15.9009
good	114.000	80.000	62.000	89.000
fool	36.000	58.000	1.000	4.000
wit	20.000	15.000	2.000	3.000

Norma L2 por documento

Documento	$\|d\|_2$
As You Like It	121.2167
Twelfth Night	99.9450
Julius Caesar	62.6283
Henry V	90.5475

Matriz TF-IDF normalizada (cada columna unitaria)

	AYLI	TN	JC	HV
battle	0.0101	0.0000	0.1367	0.1756
good	0.9405	0.8004	0.9900	0.9829
fool	0.2970	0.5803	0.0160	0.0442
wit	0.1650	0.1501	0.0319	0.0331

4.3 Similitud coseno con TF-IDF

	AYLI	TN	JC	HV
AYLI	1.0000	0.9499	0.9424	0.9448
TN	0.9499	1.0000	0.8065	0.8174
JC	0.9424	0.8065	1.0000	**0.9988**
HV	0.9448	0.8174	**0.9988**	1.0000

> ***Novela más similar a*** `Henry V` ***(TF-IDF):*** `Julius Caesar` *con* ***0.9988****.*

4.4 Similitud coseno con frecuencias crudas

L2-norms crudas:

Documento	$\|d\|_2$
AYLI	121.2147
TN	99.9450
JC	62.4340
HV	90.0833

Matriz de similitud:

	AYLI	TN	JC	HV
AYLI	1.0000	0.9499	0.9449	0.9490
TN	0.9499	1.0000	0.8090	0.8216
JC	0.9449	0.8090	1.0000	**0.9991**
HV	0.9490	0.8216	**0.9991**	1.0000

> ***Novela más similar a*** `Henry V` ***(raw):*** `Julius Caesar` *con* ***0.9991*** *(≈ 0.999, valor citado en el enunciado del quiz).*

4.5 Comparativa raw vs TF-IDF y discusión

Tabla comparada `Henry V` vs resto

Otra obra	cos (raw)	cos (TF-IDF)	Δ = tfidf – raw
As You Like It	0.9490	0.9448	–0.0042
Twelfth Night	0.8216	0.8174	–0.0042
Julius Caesar	0.9991	0.9988	–0.0003

El **orden** no cambia (`JC > AYLI > TN`), pero los valores absolutos bajan ligeramente con TF-IDF. La diferencia es minúscula porque tres de cuatro términos están en **todos** los documentos ⇒ su idf suavizada es 1 ⇒ no son penalizados realmente.

¿Qué corrige TF-IDF? ¿Qué penaliza?

TF-IDF **multiplica** la frecuencia local de un término por el logaritmo del recíproco de su frecuencia documental:

$$w_{t,d} = \text{tf}(t,d) \cdot \log \frac{N}{df(t)}$$

- **Corrige** el sesgo de las frecuencias crudas a favor de palabras **muy comunes** que aparecen en casi todos los documentos. Sin IDF, una palabra como `good`, presente en cualquier texto en inglés, distorsiona la geometría del espacio porque su magnitud bruta es enorme y arrastra el coseno hacia 1.
- **Penaliza** los términos con **alto df** (cercano a N): pierden peso relativo al cocientar entre el número de documentos donde aparecen. En el extremo, un término en todos los documentos recibe $\text{idf} = 0$ (clásica) o $\text{idf} = 1$ (suavizada) — el menor refuerzo posible.
- **Premia** los términos con **bajo df** (idealmente, presentes solo en 1 documento): obtienen idf alta y se vuelven *firmas* del documento. En este corpus, `battle` (df=3) es el único término con idf > 1 y, en consecuencia, el único cuyo peso cambia entre `raw` y `tf-idf`.

¿Por qué `Henry V ≈ Julius Caesar = 0.999` con frecuencias crudas?

Tres factores se combinan:

1. **Dominancia de** `good` en la norma L2. Ambas obras tienen `good` como su componente máxima (89 y 62), seguidas a gran distancia por el resto. La proporción $\text{good}/\|d\|$ es ≈ 0.988 (HV) y ≈ 0.993 (JC). Los vectores quedan prácticamente paralelos al eje `good`.
2. **Misma "forma" del perfil**. Tras `good`, ambas obras siguen la jerarquía `battle > fool > wit`, mientras que `AYLI` y `TN` siguen `good > fool > wit > battle`. El patrón de dominancia es el mismo en HV y JC.
3. `battle` **aporta poco al coseno crudo**. Aunque `battle` distingue cualitativamente las obras históricas/romanas (HV=13, JC=7) de las comedias (AYLI=1, TN=0), su magnitud bruta es marginal frente al `good`. Sin la ponderación IDF que **eleva** la contribución de `battle`, el coseno colapsa a la similitud impuesta por `good`.

Formalmente, con vectores $a = (a_1, \ldots)$ y $b = (b_1, \ldots)$ donde a_2, b_2 dominan:

$$\cos(a, b) \approx \frac{a_2 b_2}{\sqrt{a_2^2}\sqrt{b_2^2}} = 1$$

— el coseno se vuelve insensible al resto de dimensiones.

¿Por qué TF-IDF no logra invertir el ranking aquí?

Porque la idf de `battle` (1.22) es solo modestamente mayor que la de `good` (1.00). Con un corpus más grande y un vocabulario más diverso, donde términos discriminativos tendrían `df` mucho menor que N, la ponderación TF-IDF sí cambia el ranking. **Lección**: TF-IDF es una herramienta de *re-pesado*, no una bala de plata; su poder corrector escala con la dispersión del vocabulario en el corpus.

4.6 Reducción de dimensionalidad (PCA 2D)

PCA centrado sobre las **columnas** (documentos) de $X \in \mathbb{R}^{4\times 4}$.

Espacio TF-IDF + L2

Varianza explicada: **PC1 ≈ 96.4 %**, **PC2 ≈ 3.4 %** (acumulado ≈ 99.8 %).

Doc	PC1	PC2
AYLI	+0.087	−0.080
TN	+0.379	+0.041
JC	−0.241	+0.003
HV	−0.225	+0.035

`HV` y `JC` quedan **a la izquierda** (PC1 negativo) y muy cerca entre sí; `TN` queda al extremo derecho (alto `fool`, sin `battle`); `AYLI` queda en posición intermedia. La separación PC1 codifica el eje `battle/good ↔ fool`.

Espacio crudo (raw) + L2

Varianza explicada: **PC1 ≈ 96.8 %**, **PC2 ≈ 3.0 %** (acumulado ≈ 99.8 %).

Doc	PC1	PC2
AYLI	+0.082	+0.075
TN	+0.378	−0.038
JC	−0.240	−0.006
HV	−0.220	−0.030

Topología cualitativa idéntica al caso TF-IDF: `HV` ≈ `JC` cerca, `AYLI` intermedia, `TN` lejos. La diferencia entre los dos PCAs es marginal porque la matriz `idf` solo modificó la fila `battle`.

4.7 Conclusiones del caso

1. **TF-IDF re-pesa, no re-ordena en corpora pequeños**. Si todos los términos están en casi todos los documentos, IDF ≈ constante y la geometría apenas cambia. El método se vuelve potente con vocabularios grandes y dispersos.
2. **La normalización L2 es no negociable**. Sin ella, los documentos largos dominan por magnitud; el coseno deja de medir solo dirección.
3. `Henry V` **y** `Julius Caesar` **son las obras más similares** bajo ambos esquemas, lo cual es **lingüísticamente correcto**: las dos son obras históricas/de guerra romana, con `battle` y `good` como términos relevantes. La similitud no es un artefacto del método sino una propiedad del corpus.
4. **PCA confirma la similitud** geométricamente: `HV` y `JC` se proyectan al mismo cuadrante;

`TN` queda separada por su carga inusual de `fool`.

5. **La ley de cosenos es el primer paso**, no el último. Modelos posteriores (LSA, word2vec, BERT) profundizan esta misma idea con vectores aprendidos y contextuales.

4.8 Limitaciones del experimento

- **Corpus minúsculo**: 4 documentos × 4 términos no representa ningún corpus realista; los resultados deben leerse como demostración pedagógica del cálculo, no como conclusión literaria.
- **Sin pre-procesamiento**: no se aplica stemming, stop-words ni lematización (irrelevante con vocabulario tan reducido).
- **IDF clásica degenerada**: con $df = N$ para 3 de 4 términos, $\log_{10}(N/df) = 0$ anula esas dimensiones. Usar suavizada es decisión justificada.

Parte III — Semántica vectorial escalada

Capítulo 5. Embeddings densos y modelos contextuales

Complemento académico que extiende la perspectiva clásica del Capítulo 3 hacia las representaciones modernas: regresión logística como bloque clasificador, Word2Vec y su algoritmo en detalle, GloVe, fastText y embeddings contextuales (BERT, SBERT).

5.1 Regresión logística en PLN

La regresión logística es el **clasificador discriminativo de referencia** en PLN clásico. Aparece como bloque base en:

- Clasificación de texto (spam, sentimiento, intención).
- Identificación de idioma.
- Etiquetado de relevancia en IR.
- Capas de salida de modelos neuronales (softmax = generalización multiclase).

Modelo

Dado un vector de features $x \in \mathbb{R}^n$ (típicamente TF-IDF o un embedding promedio):

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

- $w \in \mathbb{R}^n$: pesos aprendidos por término/dimensión.
- $b \in \mathbb{R}$: sesgo.
- σ: función logística (squash a [0,1]).

Entrenamiento

Maximización de la **log-verosimilitud condicional** (equivalente a minimizar *cross-entropy*):

$$\mathcal{L}(w) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right] + \lambda$$

Optimización: SGD, L-BFGS o liblinear. La regularización L2 (λ) evita overfitting cuando $n \gg N$.

Multiclase: softmax

Para K clases:

$$P(y = k \mid x) = \frac{e^{w_k^\top x}}{\sum_{j=1}^{K} e^{w_j^\top x}}$$

Esta es la capa terminal de Word2Vec (CBOW/Skip-gram con softmax completo) y de prácticamente todo clasificador neuronal.

Conexión con la semántica vectorial

Si $x = \text{TF-IDF}(d)$, la regresión logística aprende qué términos correlacionan con cada clase — los pesos $w_k[i]$ son interpretables como "evidencia del término i a favor de la clase k". Para problemas con vocabulario alto-dimensional y poca data, suele superar a redes profundas por su sesgo inductivo lineal y su regularización fuerte.

5.2 Word2Vec: el cambio de paradigma

Mikolov *et al.* (2013) — *"Efficient Estimation of Word Representations in Vector Space"* (Google). Reemplazó la matriz dispersa de TF-IDF por **vectores densos** (~100–300 dim) aprendidos por una red shallow que predice contextos.

Dos arquitecturas

Variante	Predice	Mejor para
CBOW (Continuous Bag-of-Words)	la palabra dado su contexto	corpora pequeños, palabras frecuentes
Skip-gram	el contexto dado la palabra	corpora grandes, palabras raras

Skip-gram en detalle

Dado un par (w_t, w_{t+j}) con $j \in [-c, c] \setminus \{0\}$ (ventana de contexto c):

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\neq 0} \log P(w_{t+j} \mid w_t)$$

Con softmax pleno:

$$P(w_O \mid w_I) = \frac{\exp({v'_{w_O}}^{\top} v_{w_I})}{\sum_{w=1}^{|V|}\exp({v'_w}^{\top} v_{w_I})}$$

Cada palabra tiene **dos** vectores: v_w (input) y v'_w (output). Tras entrenar se usa solo v_w o la suma.

Negative Sampling — el truco que lo hace viable

Softmax con $|V| \approx 10^6$ es prohibitivo. Mikolov *et al.* reemplazan el denominador por una muestra de k "palabras negativas" (no aparecen en el contexto real):

$$\log \sigma({v'_{w_O}}^\top v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-{v'_{w_i}}^\top v_{w_I}) \right]$$

- $P_n(w) \propto U(w)^{3/4}$: distribución de unigramas suavizada.
- $k = 5$–20 para corpora pequeños, 2–5 para grandes.
- Reduce complejidad de $O(|V|)$ a $O(k)$ por paso.

Subsampling de palabras frecuentes

Cada palabra w se descarta del corpus con probabilidad:

$$P_{drop}(w) = 1 - \sqrt{\frac{t}{f(w)}}$$

con $t \approx 10^{-5}$. Elimina ruido de `the`, `of`, `and` — análogo al efecto de IDF en TF-IDF.

Propiedades emergentes

Las direcciones del espacio capturan **relaciones lineales**:

```
king − man + woman ≈ queen
Paris − France + Italy ≈ Rome
```

Este comportamiento (analogías por aritmética vectorial) fue la prueba empírica que detonó la transición de bag-of-words a embeddings densos.

5.3 Word2Vec con Gensim

Instalación y carga

```
!pip install gensim
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
```

Entrenamiento desde cero

```
sentences = [simple_preprocess(line) for
        line in corpus_lines]

model = Word2Vec(
    sentences=sentences,
    vector_size=100,    # dim del embedding
    window=5,           # ventana de
        contexto
    min_count=5,        # ignora palabras
        con freq < 5
    sg=1,               # 0=CBOW, 1=Skip-
        gram
    negative=10,        # k de negative
        sampling
    sample=1e-3,        # subsampling
        threshold
    epochs=10,
    workers=4,
)
model.save('w2v.model')
```

Consultas típicas

```
model.wv['battle']
        # vector (100,)
model.wv.most_similar('battle', topn=5)
        # vecinos por coseno
model.wv.similarity('battle', 'war')
        # similitud coseno
model.wv.doesnt_match(['battle','war','peac
        e','sword'])  # outlier
model.wv.most_similar(positive=
        ['king','woman'], negative=['man'])
        # analogía
```

Cargar embeddings preentrenados

```
import gensim.downloader as api
wv = api.load('word2vec-google-news-300')
        # 3.6 GB, 3M tokens
wv = api.load('glove-wiki-gigaword-100')
        # 128 MB
```

Vector de documento

Para representar un documento, lo más simple es **promedio** de embeddings de sus palabras:

```
import numpy as np
def doc_vec(tokens, wv):
    vs = [wv[t] for t in tokens if t in wv]
    return np.mean(vs, axis=0) if vs else
        np.zeros(wv.vector_size)
```

Alternativas: SIF (Arora 2017), weighted average por IDF, *doc2vec* (Le & Mikolov 2014).

5.4 GloVe: Global Vectors

Pennington, Socher, Manning (2014) — Stanford. GloVe combina lo mejor de los modelos basados en **conteo global** (LSA) y los **predictivos locales** (Word2Vec).

Idea central

Aprende embeddings tal que el **producto punto reproduzca el log-ratio de probabilidades de co-ocurrencia**:

$$w_i^\top \tilde{w}_j + b_i + \tilde{b}_j = \log X_{ij}$$

donde X_{ij} es la frecuencia con que i y j co-ocurren dentro de una ventana en el corpus.

Función de pérdida

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

con $f(x) = \min((x/x_{max})^\alpha, 1)$ (típicamente $\alpha = 0.75$, $x_{max} = 100$) para no dejar que pares ultra-frecuentes dominen.

Word2Vec vs GloVe

Aspecto	Word2Vec	GloVe
Señal	Pares locales (ventana)	Estadística global (matriz co-oc)
Entrenamiento	SGD streaming	Optimización sobre matriz pre-computada
Memoria	Baja	Alta (matriz $\|V\|^2$ dispersa)
Empíricamente	Mejor en analogías sintácticas	Mejor en similitudes semánticas

Uso

GloVe publica embeddings preentrenados (Wikipedia + Gigaword, Common Crawl, Twitter):

```
import gensim.downloader as api
glove = api.load('glove-wiki-gigaword-300')
glove.most_similar('shakespeare', topn=5)
```

5.5 fastText

Bojanowski, Grave, Joulin, Mikolov (2017) — Facebook AI. Extiende Word2Vec representando cada palabra como **suma de n-gramas de caracteres**.

Por qué subwords

Word2Vec/GloVe asignan un vector único por palabra → no funciona con:

- **OOV** (palabras no vistas durante entrenamiento).
- **Morfología rica** (español, alemán, finés, turco): `caminar`, `caminé`, `caminaríamos` se aprenden por separado.

Representación

Una palabra w se descompone en n-gramas con $n \in [3, 6]$, rodeada por `<` y `>`:

```
where → <wh, whe, her, ere, re>, <where>
```

El embedding de w es la suma de embeddings de sus n-gramas:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

Ventajas

- **OOV**: una palabra nueva se compone de sus n-gramas — siempre hay vector.
- **Morfología**: `caminar` y `caminé` comparten n-gramas → vectores cercanos sin entrenamiento explícito.
- Entrenamiento similar a Skip-gram + negative sampling.

fastText supervisado

Además de embeddings, fastText incluye un clasificador de texto extremadamente rápido (lineal sobre el promedio de embeddings) que en muchos benchmarks iguala a redes profundas con $1000\times$ menos cómputo.

Uso

```
from gensim.models import FastText
ft = FastText(sentences, vector_size=100,
        window=5, min_count=5, sg=1)
ft.wv['caminaríamos']   # funciona aunque
        no esté en el vocab
```

5.6 Embeddings profundos y de sentencias

Limitación de embeddings estáticos

Word2Vec/GloVe/fastText asignan **un único vector por palabra** independiente del contexto. Pero `banco` ≠ `banco` (institución financiera vs. asiento). Se requieren embeddings **contextuales**.

ELMo (2018)

Peters *et al.* — *Deep Contextualized Word Representations*. BiLSTM bidireccional preentrenado sobre Language Modeling. Cada palabra recibe vector dependiente de la oración completa.

BERT (2018)

Devlin *et al.* (Google) — *Bidirectional Encoder Representations from Transformers*. Marcó la transición a la era Transformer en PLN.

Componente	Detalle
Arquitectura	Stack de encoders Transformer (12 o 24 capas)
Pre-training	**MLM** (predecir tokens enmascarados) + **NSP** (frase siguiente)
Tokens especiales	`[CLS]` (clasificación), `[SEP]` (separador)
Tokenización	WordPiece
Salida	Embedding contextual por token (768 o 1024 dim)

El embedding del token `[CLS]` se usa como representación de la oración completa para clasificación.

Sentence-BERT (SBERT, 2019)

Reimers & Gurevych — siamese network sobre BERT entrenado con triplet loss / contrastive loss sobre pares de oraciones. Produce embeddings de **oración** comparables por coseno sin necesidad de pasar pares por el modelo (104× más rápido que BERT crudo en STS).

```
from sentence_transformers import
        SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-
        v2')
embs = model.encode(['Henry V is about
        war', 'Twelfth Night is a comedy'])
from sklearn.metrics.pairwise import
        cosine_similarity
cosine_similarity([embs[0]], [embs[1]])
```

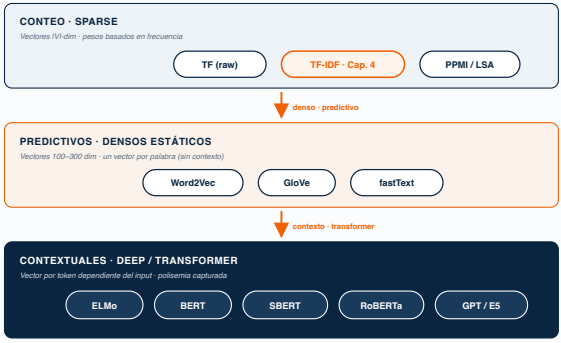
Familia post-BERT relevante

Modelo	Año	Aporte
RoBERTa	2019	BERT mejor entrenado, sin NSP
ALBERT	2019	Parámetros compartidos, menor footprint
DistilBERT	2019	Destilación, 40% más chico y 60% más rápido
GPT-3 / 4	2020+	LM autoregresivos a escala; embeddings vía API
E5, BGE, MPNet	2022+	SBERT-style optimizados para recuperación

Cuándo usar qué

Caso	Recomendación
Pocas oraciones, vocabulario fijo, baseline	**TF-IDF + coseno** (Capítulo 4)
Similitud de palabras, idioma morfológico	**fastText**
Similitud semántica, recursos limitados	**GloVe / Word2Vec preentrenado**
Clasificación sentence-level con poca data	**SBERT + logistic regression**
Búsqueda semántica, RAG	**SBERT / E5 / BGE** + índice FAISS
Clasificación SOTA, recursos abundantes	**Fine-tune BERT/RoBERTa**

5.7 Continuum conceptual



Continuum de representaciones del lenguaje: cada salto resuelve una limitación del paso anterior.

Cada salto resuelve una limitación del anterior:

1. **Raw → TF-IDF**: corregir el sesgo de palabras frecuentes (Capítulo 4).
2. **TF-IDF → Word2Vec**: pasar de vectores dispersos $|V|$-dim a densos 100-dim, capturar relaciones semánticas.
3. **Word2Vec → fastText**: manejar OOV y morfología.
4. **Estático → BERT**: capturar polisemia y contexto.
5. **BERT → SBERT**: producir embeddings de oración comparables eficientemente.

Conclusiones integradoras

Las dos actividades cubiertas en este documento responden, juntas, a una pregunta práctica única: **¿cómo convertir texto en estructuras numéricas comparables por geometría?** La respuesta tiene dos etapas — segmentación y ponderación — y este documento ha recorrido ambas.

Tokenización (Parte I). SentencePiece unifica BPE, WordPiece, Unigram, Word y Char bajo una misma API. Sobre un subset de 200 000 oraciones del *Spanish Billion Words Corpus*, el experimento entrenó 13 modelos (1 base + 12 combinaciones de algoritmo × vocab) y midió compresión sobre cinco sentencias de prueba. El equilibrio práctico para español queda en **BPE con vocab 20k–30k** (≈ 50 tokens para las 5 frases), mientras que `char` paga un factor 7–8× en longitud sin ganar nada en cobertura y `word` es competitivo solo cuando se acepta el riesgo de OOV. Los símbolos especiales (`<sep>` , `<cls>`) se incorporan limpiamente vía `user_defined_symbols` , replicando el esquema estilo BERT sin postprocesado.

Semántica vectorial clásica (Parte II). Sobre el dataset de Jurafsky & Martin (cuatro obras de Shakespeare × cuatro términos), TF-IDF con normalización L2 sienta la representación; la ley de cosenos entrega la métrica; PCA visualiza la topología. El hallazgo central es que `Henry V ≈ Julius Caesar ≈ 0.999` bajo ambas ponderaciones (raw y TF-IDF) porque el término `good` domina la norma L2 de ambos vectores. TF-IDF eleva el peso de `battle` (único término con `df < N`) pero el corpus es demasiado pequeño y poco disperso para invertir el ranking — una lección sobre los límites del re-pesado.

Semántica vectorial escalada (Parte III). Word2Vec, GloVe y fastText reemplazan vectores dispersos `|V|`-dim por densos `100`-dim entrenados sobre relaciones de co-ocurrencia. BERT y SBERT añaden contextualización: el mismo token recibe vector distinto según la oración. La regresión logística sigue siendo el clasificador discriminativo de referencia, ya sea sobre TF-IDF o sobre embeddings densos, y aparece como capa terminal de prácticamente todo modelo neuronal supervisado.

El **continuum** completo va de los conteos crudos a los Transformers, y cada salto resuelve

una limitación del paso anterior. Los métodos clásicos (TF-IDF, coseno, PCA, regresión logística) no quedan obsoletos: siguen siendo el baseline obligado por su interpretabilidad, su robustez con poca data y su costo computacional bajo. Los métodos modernos (BERT, SBERT) ganan en escenarios con vocabulario abierto, polisemia y grandes volúmenes de datos disponibles. Conocer ambos extremos del continuum es lo que distingue al ingeniero de PLN del usuario de APIs.

Referencias consolidadas

Tokenización

1. Sennrich, R., Haddow, B., Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL.
2. Schuster, M., Nakajima, K. (2012). *Japanese and Korean voice search*. ICASSP.
3. Kudo, T. (2018). *Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates*. ACL.
4. Kudo, T., Richardson, J. (2018). *SentencePiece: A simple and language independent subword tokenizer*. EMNLP demo.
5. HuggingFace Tokenizers — https://huggingface.co/docs/tokenizers.
6. SentencePiece repo — https://github.com/google/sentencepiece.

Semántica vectorial clásica

7. Harris, Z. (1954). *Distributional structure*. Word 10(2–3).

8. Salton, G., Wong, A., Yang, C. S. (1975). *A Vector Space Model for Automatic Indexing*. CACM 18(11).
9. Spärck Jones, K. (1972). *A Statistical Interpretation of Term Specificity*. Journal of Documentation.
10. Salton, G., Buckley, C. (1988). *Term-weighting Approaches in Automatic Text Retrieval*. IPM 24(5).
11. Pearson, K. (1901). *On Lines and Planes of Closest Fit to Systems of Points in Space*. Phil. Mag.
12. van der Maaten, L., Hinton, G. (2008). *Visualizing Data using t-SNE*. JMLR.
13. Manning, C., Raghavan, P., Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge UP — Cap. 6.

Semántica vectorial escalada

14. Mikolov, T., Chen, K., Corrado, G., Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. ICLR Workshop.
15. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J. (2013). *Distributed*

Representations of Words and Phrases and their Compositionality. NeurIPS.

16. Pennington, J., Socher, R., Manning, C. (2014). *GloVe: Global Vectors for Word Representation*. EMNLP.
17. Bojanowski, P., Grave, E., Joulin, A., Mikolov, T. (2017). *Enriching Word Vectors with Subword Information*. TACL.
18. Peters, M. *et al.* (2018). *Deep Contextualized Word Representations*. NAACL.
19. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.
20. Reimers, N., Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP.
21. Řehůřek, R. (2010). *Software Framework for Topic Modelling with Large Corpora — Gensim*. LREC Workshop.

Referencias transversales

22. Jurafsky, D., Martin, J. (2024). *Speech and Language Processing* (3rd ed. draft), Caps. 5 (Logistic Regression), 6 (Vector Semantics),

11 (BERT).
https://web.stanford.edu/~jurafsky/slp3/

23. Goldberg, Y. (2017). *Neural Network Methods for Natural Language Processing*. Morgan & Claypool.